CSC10014 · PRACTICAL TRACK · 2026

Lab 4

Your First Web API

HTTP, JSON and FastAPI from zero — then the study assistant as a web service

Practical Track · Week 4 · 3 h · No prior web experience needed

Faculty of Information Technology · University of Science, VNU-HCM

Instructor: Dr. LE Thanh Tung

Contents

About This Lab

Lab 4 — Your First Web API. Practical track, Week 4. Duration: 3 hours in class, plus a short homework.

This lab assumes you know **basic programming, DSA and OOP** — and **nothing about the web**. Every new word is explained the first time it appears. If something is unclear, it is the tutorial's fault, not yours: ask.

What you already know → what is new today

You already know	Today you learn	The link
calling a function `f(x)` and getting a result	sending a **request** to a server and getting a **response**	a web API is a function you call over the network
a Python `dict`	**JSON**, the text format for data on the web	JSON looks almost exactly like a dict
`dict` mapping keys to values	a **route** mapping a URL path to a function	FastAPI keeps a table: path → function
raising an exception	an HTTP **status code** such as 404	the web's way of saying "something went wrong"
classes with typed fields (OOP)	**Pydantic models** that check the data you receive	a class describes what valid input looks like

Goal

By the end, the study assistant from Lab 3 can be used **over HTTP**: a browser, a phone app or a test can send it a question and get an answer back as JSON.



One contract for every caller: the UI, tests and other services all speak HTTP + JSON.

Figure 0.1 — The journey of one request. Do not worry about the details yet; we build it step by step.

What you hand in

Level	Task	Where
Must	warm-up P1–P5 (`practice/hello_api.py`) — all tests green	your branch
Must	project tasks T1–T4 (`starter/`) — all tests green	your branch, PR

Level	Task	Where
Must	one screenshot of `/docs` with a successful `/v1/chat` call	in the PR
Should	homework H1: add a `/v1/offices` endpoint with two tests	PR
Bonus	rate limiting (Part F) or your own external API	PR

Timeline

Time	Activity
0:00–0:30	Part A: how the web works — no code yet
0:30–0:45	Part B: JSON in five minutes
0:45–1:30	Part C: warm-up — your first API (P1–P5)
1:30–1:45	break
1:45–2:30	Part D: the assistant API (T1–T4)
2:30–2:50	Part E: testing with Swagger, curl and pytest
2:50–3:00	submit; explain the homework

Before Class

1. Lab 3 works on your laptop: `pytest -q` passes in your `assistant_core` project.
2. Download the Lab 4 materials and unzip them. You will use `lab04/practice/` and `lab04/starter/`.
3. Optional reading (20 minutes): the first two pages of the FastAPI tutorial, *First Steps*.

Set up once:

```
$ cd lab04/starter
$ python -m venv .venv
$ source .venv/bin/activate            # Windows: .venv\Scripts\activate
(.venv) $ pip install -r requirements.txt
(.venv) $ pip install -e .
(.venv) $ cp .env.example .env
```

The same `.venv` works for `lab04/practice/` too.

Part A — How the Web Works (No Code Yet)

A.1 A restaurant analogy

Restaurant	Web
You (the customer)	the **client** — a browser, a phone app, a test program
The waiter takes your order	the **request** travels over the network
The kitchen cooks	the **server** — a program waiting for requests
The menu	the **API** — the list of things you may ask for
The dish arrives	the **response**
"Sorry, that dish is sold out"	a **status code** such as 404 or 503

The client and server are just two programs. They can run on two computers far apart — or both on your laptop, which is what we do today.

A.2 A URL, piece by piece

```
http://127.0.0.1:8000/v1/chat?lang=en
scheme = http        host = 127.0.0.1      port = 8000
path   = /v1/chat    query = lang=en
```

Part	Meaning	Today
scheme	how to talk: `http` or `https` (encrypted)	`http` on your laptop
host	which computer	`127.0.0.1` = "this computer" (also called `localhost`)
port	which program on that computer	`8000`
path	which "dish" on the menu	`/v1/chat`
query	optional extra options after `?`	`?lang=en`

A.3 What a request and a response look like

They are plain text. You will rarely write them by hand, but it helps to see one:

```
POST /v1/chat HTTP/1.1                    <- method + path
Host: 127.0.0.1:8000
Content-Type: application/json             <- "the body is JSON"

{"message": "where is the library"}        <- the body (the data we send)
```

```
HTTP/1.1 200 OK                            <- status code: 200 means success
Content-Type: application/json

{"answer": "Library: room B.201, open Mon-Sat 07:00-20:00.", "intent": "office"}
```

A.4 Methods: what kind of request?

For now you only need two:

Method	Meaning	Like in Python
GET	"give me something" — reading only	calling a getter
POST	"here is some data, do something with it"	calling a function with arguments

A.5 Status codes: how did it go?

Code	Meaning	Everyday meaning
200	OK	here is your answer
201	Created	I saved what you sent
404	Not Found	that does not exist
422	Unprocessable	your data is wrong (e.g. empty message)
401	Unauthorized	you need a key for this
500	Server Error	the server has a bug
503	Service Unavailable	something the server depends on is down

> **TIP**
> Codes starting with **2** = success, **4** = the *client* made a mistake, **5** = the *server* has a problem.

> **CHECK YOURSELF**
> 1) In `http://127.0.0.1:8000/hello/Lan`, what is the path?
> 2) Which method would you use to *send* feedback?
> 3) A user sends an empty question. Which status code fits?

Part B — JSON in Five Minutes

JSON is how programs exchange data on the web. It looks almost exactly like a Python dict:

```
$ python
>>> import json
>>> answer = {"answer": "Room B.201", "intent": "office", "sources": [], "degraded": False}
>>> text = json.dumps(answer)          # Python -> JSON text
>>> text
'{"answer": "Room B.201", "intent": "office", "sources": [], "degraded": false}'
>>> back = json.loads(text)            # JSON text -> Python
>>> back["answer"]
'Room B.201'
```

Python	JSON
`dict`	object `{...}`
`list`	array `[...]`
`str`	string (always double quotes `"..."`)
`int`, `float`	number
`True` / `False`	`true` / `false`
`None`	`null`

> **NOTE**
> FastAPI converts between Python and JSON for you. You return a dict or an object; the client receives JSON.

Part C — Warm-up: Your First API

Open `lab04/practice/hello_api.py`. It is a complete tiny web API with five small exercises (P1–P5).

C.1 Run it

```
(.venv) $ cd lab04/practice
(.venv) $ uvicorn hello_api:app --reload
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

`uvicorn` is the **server** program. `hello_api:app` means "the variable `app` in the file `hello_api.py`". `--reload` restarts the server whenever you save the file. Leave this terminal running and open a **second** terminal for other commands.

Open your browser at **http://127.0.0.1:8000/** — you should see:

```
{"message":"Hello, CSC10014!"}
```

You just made a request (the browser is the client) and got a JSON response.

C.2 Read the first endpoint

```
app = FastAPI(title="Hello API")

@app.get("/")
def home() -> dict:
    return {"message": "Hello, CSC10014!"}
```

- `@app.get("/")` is a **decorator**: it registers the function `home` for GET requests to path `/`. Think of FastAPI keeping a dict `{("GET", "/"): home, ...}`.
- The function returns a dict; FastAPI sends it as JSON.

C.3 The magic page: /docs

Open **http://127.0.0.1:8000/docs**. FastAPI generates an interactive page listing every endpoint. Click an endpoint → **Try it out** → **Execute**. You will use this page all the time to test without writing any client.

C.4 Exercises P1–P5

Each exercise needs 1–3 lines where the file says `TODO`. After each one, save the file and try it in the browser or in `/docs`.

#	What to do	Try it	Expected
P1	nothing — read it	`/`	`{"message":"Hello, CSC10014!"}`
P2	return a greeting with the name from the path	`/hello/Lan`	`{"message":"Hello, Lan!"}`

#	What to do	Try it	Expected
P3	return the square of `n` from the query	`/square?n=7`	`{"n":7,"square":49}`
P4	add two numbers sent as JSON	POST `/add` with `{"a":2,"b":3}` (use `/docs`)	`{"sum":5.0}`
P5	return 404 for an unknown course	`/courses/XYZ`	status 404

P2 — a path parameter

```
@app.get("/hello/{name}")
def hello(name: str) -> dict:
    return {"message": f"Hello, {name}!"}
```

The `{name}` part of the path becomes the function argument `name`.

P3 — a query parameter

```
@app.get("/square")
def square(n: int) -> dict:
    return {"n": n, "square": n * n}
```

`n` is not in the path, so FastAPI reads it from the query `?n=7`. Because of the type hint `int`, it converts `"7"` to `7` — and if you visit `/square?n=abc`, it answers **422** automatically. Try it!

P4 — a request body with a Pydantic model

```
class Numbers(BaseModel):
    a: float
    b: float

@app.post("/add")
def add(nums: Numbers) -> dict:
    return {"sum": nums.a + nums.b}
```

`Numbers` is a class describing valid input — like a C++ struct with checks built in. FastAPI reads the JSON body, checks that `a` and `b` are numbers, and gives you an object. Test it in `/docs`: click **POST /add** → **Try it out**, type `{"a": 2, "b": 3}`, **Execute**.

P5 — returning an error

```
@app.get("/courses/{code}")
def course(code: str) -> dict:
    if code not in COURSES:
        raise HTTPException(status_code=404, detail="Course not found")
    return {"code": code, "name": COURSES[code]}
```

`raise HTTPException(...)` stops the function and sends that status code to the client — the web version of raising an exception.

C.5 Check yourself with the tests

```
(.venv) $ pytest -q test_hello_api.py
.....                                                                    [100%]
5 passed
```

Read `test_hello_api.py`: each test sends a request and checks the response. You will see tests like these again in Part E.

EXERCISE

When all five tests pass, commit: `git add practice && git commit -m "feat(practice): hello API P1-P5"`.

Part D — The Assistant API

Now the real thing. Stop the warm-up server (Ctrl+C) and go to the starter:

```
(.venv) $ cd ../starter
(.venv) $ uvicorn assistant_api.main:create_app --factory --reload
```

> **NOTE**
> `--factory` tells uvicorn that `create_app` is a function that *builds* the app. This lets tests build the app with fake parts (Part E).

Open **http://127.0.0.1:8000/docs**. You see five endpoints. Only `/health` works yet — the others raise `NotImplementedError` until you finish T1–T4.

D.1 What is already given

Open `src/assistant_api/main.py` and `src/assistant_api/schemas.py`. Most of the code is written for you. Read it with this map:

Piece	What it does
`schemas.py`	Pydantic classes describing requests and responses — the **contract**
`error(...)` + two exception handlers	make every error look the same: `{"error": {"code": ..., "message": ...}}`
`require_admin`	checks the `X-API-Key` header for admin-only endpoints
`/health`	returns `{"status": "ok"}` — used to check the server is alive
`WeatherClient` (in `weather.py`)	calls a real weather website, with a time limit and a backup plan

D.2 The contract

Before coding, know what each endpoint must do:

Method	Path	You send	You get back
GET	`/health`	nothing	`{"status": "ok", "version": "1.0.0"}`
POST	`/v1/chat`	`{"message": "...", "session_id": "..."}`	`{"answer", "intent", "sources", "degraded", "request_id"}`
POST	`/v1/feedback`	`{"request_id": "...", "rating": -1, 0 or 1, "comment": "..."}`	201 + the saved feedback

Method	Path	You send	You get back
GET	`/v1/feedback`	header `X-API-Key: <key>`	the list of feedback, or 401
GET	`/v1/weather`	nothing	the campus weather, or 503 if the weather site is down

D.3 T1 — POST /v1/chat (2 lines)

The endpoint receives a `ChatRequest` (message + session id). It must ask the Lab 3 service for an answer and send it back with a new `request_id`.

```
    def chat(req: ChatRequest, request: Request) -> ChatResponse:
        resp = service.handle(Message(text=req.message, session_id=req.session_id))
        return ChatResponse(**resp.model_dump(), request_id=uuid.uuid4().hex[:12])
```

What these lines mean:

- `service.handle(...)` is exactly the function you called in Lab 3 — nothing new.
- `resp.model_dump()` turns the Lab 3 `Response` object into a dict; `**` passes its fields into `ChatResponse`.
- `uuid.uuid4().hex[:12]` makes a random id such as `3f9a2c1b7e0d`, so the user can later rate *this* answer.

Try it in `/docs` with the message `tuition for 18 credits and 2 retake`:

```
{"answer": "Your tuition is 9,450,000 VND.", "intent": "fee", "sources": [], "degraded":
false, "request_id": "3f9a2c1b7e0d"}
```

Now try an **empty** message. You get 422 with our error shape — you did not write any check: the `ChatRequest` model says the message needs at least one character.

D.4 T2 — POST /v1/feedback (3 lines)

For now we store feedback in a Python list, `feedback_store`. (It disappears when the server restarts; Lab 5 fixes that with a database.)

```
    def add_feedback(fb: FeedbackIn) -> FeedbackOut:
        item = FeedbackOut(**fb.model_dump(), id=len(feedback_store) + 1,
created_at=datetime.now(timezone.utc))
        feedback_store.append(item)
        return item
```

The decorator already says `status_code=201`, so the client learns "created". Try a rating of `5` — you get 422, because `FeedbackIn` allows only −1, 0 or 1.

D.5 T3 — GET /v1/feedback, admin only (1 line)

```
    def list_feedback() -> list[FeedbackOut]:
```

```
        return feedback_store
```

The protection is already there: `dependencies=[Depends(require_admin)]` runs `require_admin` *before* your function. Test in `/docs`: without a key you get 401. Fill **x-api-key** with the value of `API_ADMIN_KEY` from your `.env` (default `change-me`), and you get the list.

> **WARNING**
> A real key never appears in code or in Git — only in `.env`, which is in `.gitignore`.

D.6 T4 — GET /v1/weather (4 lines)

The weather client is given. It asks the free Open-Meteo website for the current weather at the Nguyen Van Cu campus. Your job is only to turn "the weather site is down" into a clean 503:

```
    def campus_weather() -> Weather:
        try:
            return weather.current()
        except ExternalServiceError:
            raise HTTPException(status.HTTP_503_SERVICE_UNAVAILABLE, "Weather service
unavailable") from None
```

Why the weather client is careful

Other people's websites can be slow or down. The given `WeatherClient` follows three habits you will reuse for every outside service:

Habit	In plain words	In the code
Timeout	"wait at most 3 seconds"	`httpx.Client(timeout=3)`
Retry	"if it fails, try once more"	a small `for` loop
Backup plan	"if it still fails, show the last known weather, marked old"	the cache and `stale=True`

> **NOTE**
> If the campus network blocks the weather site, `/v1/weather` returns 503. That is correct behaviour — and the tests still pass, because they use a fake weather site (Part E.4).

Part E — Testing Your API

E.1 With the browser: /docs

Use **Try it out** for every endpoint. Take a screenshot of a successful `/v1/chat` call for your PR.

E.2 With curl (from a second terminal)

`curl` sends requests from the command line. Here is one request, explained:

```
$ curl -s -X POST http://127.0.0.1:8000/v1/chat \
       -H "Content-Type: application/json" \
       -d '{"message": "where is the library"}'
```

Part	Meaning
`-X POST`	the method
`-H "Content-Type: application/json"`	a header: "my body is JSON"
`-d '...'`	the body

The script `api-tests/curl_tests.sh` sends every request in the contract and prints the status code of each:

```
$ bash api-tests/curl_tests.sh
== health (expect 200)
{"status":"ok","version":"1.0.0"}  [200]
== chat empty (expect 422)
{"error":{"code":"invalid_request","message":"message: String should have at least 1
character"}}  [422]
...
```

> **TIP**
> On Windows, run curl in **Git Bash**. PowerShell treats quotes differently. Or use `/docs` or Postman.

E.3 With Postman or Bruno (optional)

Import `api-tests/study-assistant.postman_collection.json`. Every request is already prepared — click **Send**.

E.4 With pytest — the real check

```
(.venv) $ pytest -q
.....................s                                   [100%]
23 passed, 1 skipped
```

When T1–T4 are done, every test passes (the skipped one is the bonus). How do the tests work without starting a server?

```
@pytest.fixture
def client(settings, service):
    app = create_app(settings, service, weather_with(lambda req: httpx.Response(200,
json=OK_JSON)))
    return TestClient(app)

def test_chat_validation_error(client):
    r = client.post("/v1/chat", json={"message": ""})
    assert r.status_code == 422 and r.json()["error"]["code"] == "invalid_request"
```

- `TestClient` sends requests to the app directly — no network, very fast.
- `weather_with(...)` builds a weather client whose "website" is a Python function returning fake JSON. So tests never depend on the internet.

EXERCISE

Open `tests/test_api.py` and read `test_weather_down_returns_503`. In one sentence, explain how it pretends the weather site is down.

Part F — Bonus: Rate Limiting

A rate limit stops one client from sending too many requests (for example, a bug in a phone app sending 1,000 requests per second). The file `ratelimit.py` has a small `RateLimiter`. To use it in `/v1/chat`, add at the start of the function:

```
        client = request.client.host if request.client else "unknown"
        if not limiter.allow(client):
            raise HTTPException(status.HTTP_429_TOO_MANY_REQUESTS, "Too many requests")
```

Then remove the `@pytest.mark.skip(...)` line above `test_rate_limit` and run the tests.

Team Integration (last 10 minutes or after class)

Merge **one** version of the API into the team repository:

1. Each member opens a PR with their `starter/` work.
2. The team chooses the clearest one to merge; the others review it.
3. Member 1 tags `v0.4-api`.

Homework (about 1 hour)

H1 — Add GET /v1/offices (should)

Add an endpoint that returns all offices as a list:

```
[{"name": "IT Helpdesk", "room": "E.005", "hours": "Mon-Fri 08:00-17:00"}, {"name": "Library",
...}, ...]
```

Steps:

1. In `src/assistant_core/knowledge.py`, `OfficeDirectory` keeps offices in `self._offices` (a dict). Add a method `all()` that returns the `Office` objects as a list, sorted by name.
2. In `main.py`, add `@app.get("/v1/offices")` that returns `[o.__dict__ for o in service.offices.all()]`.
3. Add two tests in `tests/test_api.py`: the status is 200, and one item has the name "Library".

H2 — Reflection (must, 5 lines)

In your own words: what is the difference between calling `service.handle()` in Lab 3 and calling `POST /v1/chat` today? Why might a team prefer the HTTP version?

H3 — Bonus

Connect one more free public API (for example, public holidays or exchange rates) using the same three habits: timeout, retry, backup plan. Use `lab04/worksheets/external_api_failure.md`.

How to Submit

What	When	How
P1–P5 + T1–T4, tests green, `/docs` screenshot	end of the session (draft PR OK)	PR link on the LMS
Team API merged, tag `v0.4-api`	23:59 the day before Lab 5	Tech Lead: tag link
H1 + H2	23:59 the day before Lab 5	PR link on the LMS

Grading checklist

Criterion	Points
P1–P5 all tests pass	2
T1–T4 all tests pass	4
`/docs` screenshot and curl script run	1
H1 `/v1/offices` with two tests	2
H2 reflection	1
Bonus (rate limit or extra API)	+1
Total	**10 (+1)**

Troubleshooting

Problem	What to do
`uvicorn: command not found`	activate `.venv`; run `pip install -r requirements.txt`
`Error loading ASGI app`	run from the folder with the file; for the starter use `--factory`
`Address already in use`	another server is running — stop it (Ctrl+C) or add `--port 8001`
Browser shows `{"detail":"Not Found"}`	check the path spelling; open `/docs` to see all paths
422 when you expected success	read the error message: it names the wrong field
500 Internal Server Error	a `TODO` still raises `NotImplementedError` — look at the server terminal
curl quote errors on Windows	use Git Bash or `/docs`
Weather always 503	the network blocks the site; that is fine, tests use a fake site

Glossary

Word	Meaning
API	a list of requests a program accepts, and what it returns
client / server	the program that asks / the program that answers
endpoint	one method + path, e.g. `POST /v1/chat`
request / response	the message sent / the message returned
JSON	text format for data, similar to a Python dict
status code	a number saying how the request went (200, 404, 422 …)
header	extra information on a request, e.g. `Content-Type`, `X-API-Key`
Pydantic model	a class that describes and checks data
uvicorn	the server program that runs a FastAPI app
`/docs`	automatic, interactive documentation of your API

Self-Check

1. Name the five parts of a URL.
2. What does status code 422 tell the client?
3. How does FastAPI know that `n` in `/square?n=7` is an integer?
4. Why does `GET /v1/feedback` need an API key?
5. Why do the tests use a fake weather website?

Answers. 1) Scheme, host, port, path, query. 2) The data sent is not valid. 3) From the type hint `n: int` — it converts and checks the value. 4) Feedback is only for staff; the key keeps other users out. 5) So tests are fast, repeatable, and work without internet.

Resources

- FastAPI tutorial — *First Steps*, *Path Parameters*, *Query Parameters*, *Request Body* (fastapi.tiangolo.com/tutorial/)
- MDN — *An overview of HTTP* (developer.mozilla.org)
- Course companion — *Git & GitHub for Team Projects*, Chapter 9 (team workflow)